

```
"""
```

dataset_fetcher.py – GRUS Audit Pipeline Stage 4

Specification: GRUS-STD-001-v1.0 Section 3 Stage 4; Clause 3

Downloads cited public datasets and computes SHA-256 hashes for the certification record. Verifies all datasets are publicly accessible.

Published by: Green Recursive Utility Service LLC

```
"""
```

```
import urllib.request, urllib.error, hashlib, os
from datetime import datetime, timezone
from typing import Dict, List
from dataclasses import dataclass, field
```

```
@dataclass
```

```
class DatasetFetchResult:
```

```
    passed: bool
    datasets: List[Dict] = field(default_factory=list)
    failure_reason: str = ""
    failing_url: str = ""
```

```
def fetch_dataset(url: str, cache_dir: str) -> Dict:
```

```
    os.makedirs(cache_dir, exist_ok=True)
    safe_name = hashlib.sha256(url.encode()).hexdigest()[:16]
    cache_path = os.path.join(cache_dir, safe_name)
    accessed_at = datetime.now(timezone.utc).isoformat()
    try:
        req = urllib.request.Request(url, headers={"User-Agent": "GRUS-Audit/1.0"})
        with urllib.request.urlopen(req, timeout=60) as response:
            content = response.read()
    except urllib.error.HTTPError as e:
        return {"url": url, "accessible": False,
                "error": f"HTTP {e.code}", "accessed_utc": accessed_at}
    except Exception as e:
        return {"url": url, "accessible": False,
                "error": str(e), "accessed_utc": accessed_at}
    sha256 = hashlib.sha256(content).hexdigest()
    with open(cache_path, "wb") as f:
        f.write(content)
    return {"url": url, "accessible": True,
            "sha256": sha256, "size_bytes": len(content),
            "accessed_utc": accessed_at, "cache_path": cache_path}
```

```
def fetch_all_datasets(manifest: Dict, cache_dir: str = "/tmp/grus_dataset_cache") ->
```

```
DatasetFetchResult:
```

```
    cited = manifest.get("datasets_cited", [])
    if not cited:
        return DatasetFetchResult(passed=False,
                                    failure_reason="No datasets cited in manifest (Clause 3 violation)")
    results = []
```

```

for d in cited:
    url = d.get("url", "")
    if not url:
        return DatasetFetchResult(passed=False,
                                   failure_reason="Dataset entry missing URL", failing_url="(none)")
    r = fetch_dataset(url, cache_dir)
    if not r["accessible"]:
        return DatasetFetchResult(passed=False, datasets=results,
                                   failure_reason=f"Dataset not accessible: {r.get('error')}",
                                   failing_url=url)
    r["description"] = d.get("description", "")
    r["access_statute"] = d.get("access_statute", "OSTP 2022 Public Access Mandate")
    results.append(r)
return DatasetFetchResult(passed=True, datasets=results)

if __name__ == "__main__":
    import sys, yaml
    if len(sys.argv) < 2:
        print("Usage: python dataset_fetcher.py <repo_path>"); sys.exit(0)
    with open(os.path.join(sys.argv[1], "GRUS_MANIFEST.yaml")) as f:
        m = yaml.safe_load(f)
    r = fetch_all_datasets(m)
    if r.passed:
        print(f"[PASS] Fetched {len(r.datasets)} datasets")
        for d in r.datasets:
            print(f"  {d['url']}  sha256={d['sha256'][:16]}...")
    else:
        print(f"[FAIL] {r.failure_reason}")
        if r.failing_url: print(f"  URL: {r.failing_url}")
        sys.exit(1)

```